

Serializare. Externalizare. Fire de Execuție

Serializarea obiectelor în Java

Ce este serializarea

Serializarea este procesul prin care un obiect Java este transformat într-un flux de octeți (byte stream), astfel încât să poată fi:

- salvat pe disc
- transmis prin rețea
- reconstruit ulterior (deserializare)

Procesul invers se numește deserializare.

Interfața Serializable

Pentru a permite serializarea, o clasă trebuie să implementeze:

```
import java.io.Serializable;

public class User implements Serializable {
    private String name;
    private int age;
}
```

Important:

- Serializable este o interfață marker (nu are metode)
- JVM-ul știe că obiectul poate fi serializat doar prin prezența ei

Exemplu complet

```
import java.io.*;

class User implements Serializable {
    private static final long serialVersionUID = 1L;

    String name;
```

```

int age;

public User(String name, int age) {
    this.name = name;
    this.age = age;
}
}

public class Main {
    public static void main(String[] args) throws Exception {
        User user = new User("Ana", 25);

        // Serializare
        ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream("user.dat"));
        oos.writeObject(user);
        oos.close();

        // Deserializare
        ObjectInputStream ois = new ObjectInputStream(new
        FileInputStream("user.dat"));
        User u = (User) ois.readObject();
        ois.close();

        System.out.println(u.name + " " + u.age);
    }
}

```

serialVersionUID

private static final long serialVersionUID = 1L;

Este un identificador unic pentru versiunea clasei.

De ce e important:

- previne erori la deserializare dacă structura clasei se schimbă
- dacă nu este definit, JVM îl generează automat (nerecomandat)

Câmpuri transient și static

transient

transient String password;

- NU este serializat

static

- aparține clasei, nu obiectului → nu este serializat

Metode personalizate de serializare

Poți controla procesul:

```
private void writeObject(ObjectOutputStream oos) throws IOException {  
    oos.defaultWriteObject();  
    oos.writeInt(age + 1);  
}  
  
private void readObject(ObjectInputStream ois) throws IOException,  
ClassNotFoundException {  
    ois.defaultReadObject();  
    age = ois.readInt() - 1;  
}
```

Probleme și limitări

- performanță relativ slabă
- vulnerabilități de securitate
- dependență de structură
- incompatibilitate între versiuni

Bune practici

- definește serialVersionUID
- evită serializarea obiectelor sensibile
- folosește transient pentru date temporare
- validează datele la deserializare

Externalizarea obiectelor (Externalizable)

Este o alternativă la Serializable care oferă control complet asupra serializării.

```
import java.io.Externalizable;
```

Diferențe față de Serializable

Caracteristică	Serializable	Externalizable
Control	automat	manual
Performanță	mai lent	mai rapid
Complexitate	mică	mare
Flexibilitate	limitată	maximă

Interfața Externalizable

```
public class User implements Externalizable {

    String name;
    int age;

    public User() {} // OBLIGATORIU

    public User(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public void writeExternal(ObjectOutput out) throws IOException {
        out.writeUTF(name);
        out.writeInt(age);
    }

    @Override
    public void readExternal(ObjectInput in) throws IOException {
```

```
    name = in.readUTF();  
    age = in.readInt();  
}  
}
```

Exemplu complet

```
ObjectOutputStream oos = new ObjectOutputStream(new  
FileOutputStream("user.dat"));  
oos.writeObject(new User("Ion", 30));  
oos.close();  
  
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("user.dat"));  
User u = (User) ois.readObject();  
ois.close();
```

Avantaje

- control total asupra formatului
- performanță mai bună
- compatibilitate mai ușor de gestionat

Dezavantaje

- cod mai complex
- responsabilitate completă pentru consistență
- trebuie constructor fără parametri

Când folosim Externalizable

- când performanța este critică
- când vrei format custom
- când lucrezi cu sisteme externe

Lucrul cu fire de execuție (Multithreading)

Un thread este o unitate de execuție independentă într-un program.

Java suportă multithreading pentru:

- performanță
- paralelism
- aplicații reactive

Crearea unui thread

```
//Varianta 1: extindere Thread
class MyThread extends Thread {
    public void run() {
        System.out.println("Rulează thread-ul");
    }
}

new MyThread().start();

//Varianta 2: Runnable (recomandată)
class MyTask implements Runnable {
    public void run() {
        System.out.println("Task executat");
    }
}

Thread t = new Thread(new MyTask());
t.start();
```

Metode importante

- start() - pornește thread-ul
- run() - conține logica
- sleep(ms) - pauză
- join() - așteaptă finalizarea altui thread

Sincronizare

Problema: acces concurrent la resurse

Exemplu problemă:

```
int counter = 0;
```

Mai multe thread-uri → rezultate incorecte

Soluție: synchronized

```
synchronized void increment() {  
    counter++;  
}  
  
//Sau:  
  
synchronized(this) {  
    counter++;  
}
```

Lock-uri

```
import java.util.concurrent.locks.ReentrantLock;  
  
ReentrantLock lock = new ReentrantLock();  
  
lock.lock();  
try {  
    // cod critic  
} finally {  
    lock.unlock();  
}
```

Volatile

```
volatile boolean running = true;
```

- asigură vizibilitate între thread-uri
- NU garantează atomicitate

Executor Framework

Alternativă modernă la Thread:

```
import java.util.concurrent.*;

ExecutorService executor = Executors.newFixedThreadPool(2);

executor.submit(() -> {
    System.out.println("Task executat");
});

executor.shutdown();
```

Callable vs Runnable

```
Callable<Integer> task = () -> {
    return 42;
};

Future<Integer> future = executor.submit(task);
System.out.println(future.get());
```

Probleme clasice

Race condition

- acces simultan necontrolat

Deadlock

- două thread-uri blocate reciproc

Starvation

- un thread nu primește resurse

Bune practici

- evită sincronizarea excesivă
- preferă ExecutorService
- folosește structuri thread-safe (ConcurrentHashMap)
- minimizează secțiunile critice

- folosește immutable objects

Exerciții

1. Creează o clasă Account care conține:

- username (String)
 - password (String)
 - balance (double)
- a) Clasa trebuie să fie serializabilă
- b) Câmpul password NU trebuie serializat
- c) La deserializare:
- a. dacă balance < 0, setează-l automat la 0
- d) Adaugă serialVersionUID

2. Creează o clasă Product:

- name (String)
 - price (double)
 - quantity (int)
- a) Cerințe:
- b) Implementează Externalizable
- c) La serializare:
- a. salvează doar name și valoarea totală (price * quantity)
- d) La deserializare:
- a. reconstruiește obiectul cu:
 - i. price = total / quantity
 - ii. quantity = 1 (default)

3. Cerință

Creează o clasă Counter:

- conține un câmp count

Cerințe:

- a) Creează 5 thread-uri
- b) Fiecare thread incrementează contorul de 1000 ori
- c) Afișează rezultatul final

4. Cerință

Creează un sistem care:

- a) Generează o listă de obiecte TaskResult
 - a. taskId
 - b. result
- b) Folosește ExecutorService cu 3 thread-uri
- c) Fiecare task:
 - a. calculează suma numerelor de la 1 la N
- d) După execuție:
 - a. serializează lista în fișier
- e) Apoi:
 - a. deserializază și afișează rezultatele